REPORT PROJECT 3

Part 1 - The Travelling Salesman Problem with GA :

Task 1 : What are the important operations of the algorithm ?

The **Genetic Algorithm (GA)** task is to try to find the shortest possible route that visits all cities exactly once by starting and ending from the first city. The main operations of the algorithm are:

1. **Initialization (Random Population Generation)**
   We create a population of candidate solutions (paths). Each one is a random permutation of the cities, starting and ending at city 1.

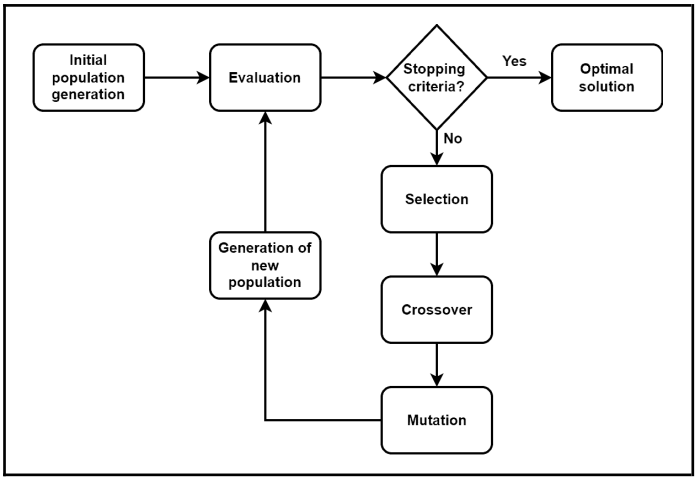
2. **Fitness Evaluation**
   To ensure that shorter routes have higher fitness values, the fitness of each path is one over the total travel distance of that path.

3. **Selection**
   Parents are selected from the population to produce offspring which will constitute the population for the next generation. Two selection methods can, and are, employed:
   - **Tournament Selection:** We choose a small subset of individuals randomly, and select the best one based on fitness.
   - **Roulette Wheel Selection:** this is a probability selection based on the fitness of each individual. it allows giving better routes a higher chance of being chosen, while still having diversity and randomness.

4. **Crossover (Ordered)**
   This operation combines two parent routes to create a new offspring. We copy a segment of the first parent to the child, fill in the remaining cities from the second parent, following the same order.

5. **Mutation (Swap Mutation Reverse)**
   We choose a random sequence of the offspring and reverse it, this improves the chances of making more significant changes, instead of a simple swap between two cities, which tends to be not effective enough. However, mutation is only applied with a certain probability, to maintain a certain level of diversity.
6. **Elitism**
   The best individuals (top X% of the population) are directly sent to the next generation, to prevent losing the best solutions.

7. **Termination Condition**
   The algorithm runs for a fixed number of iterations (generations) or stops if the number of fitness calculations surpasses the maximum fitness calculations allowed.



Task 2 : How do you represent a solution to the problem ?

A possible solution to the TSP problem is as follows:

$$X = \{X_1, X_2, \dots, X_i, X_j, \dots, X_n, X_{n+1}\}, BestFitness$$

where n is the number of cities we need to visit and $BestFitness$ is the fitness of the solution,

$$X_i \in [\![2, n]\!], \ \forall\, i \in [\![2, n]\!],$$

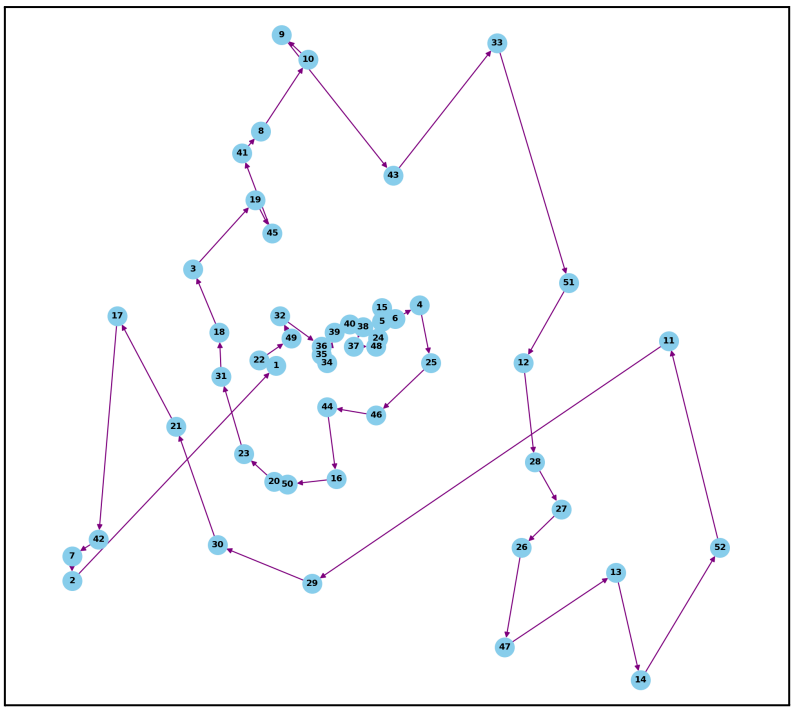
$$BestFitness = Fitness(\{X_1, X_2, \dots, X_i, X_j, \dots, X_n, X_{n+1}\})$$

where X_i represents the i-th city visited, and

$$X_1 = X_{n+1} = 1$$

since we start and end at the city number 1.

A solution is represented as an ordered sequence of nodes (cities) forming a somewhat short path from the start node (1) to the destination node (1), while visiting every other node, plus its fitness score.



The problem has the following constraint :

$$C = \{ X_i \neq X_j, \forall\, i, j \; and \; i \neq j \}$$

Task 3 : What is the equation of the fitness function ?

$$Fitness(\{ X_1, X_2, \dots, X_i, X_j, \dots, X_n, X_{n+1} \}) = \sum_{i=1}^{n} \frac{1}{d(X_i, X_{i+1})}$$

Here, $d(X_i, X_{i+1})$ is the euclidean distance $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ between two cities.

Task 4 : What are the parameters used in the algorithm ?

Population size (100): this determines the number of paths (solutions) for each generation. A larger population increases diversity but because of the **Maximum fitness evaluations allowed, it decreases the number of iterations**.

Number of iterations (1400): the algorithm runs for 1400 iterations, unless the maximum fitness calculation allowed is exceeded.

Mutation probability (0.3): each individual has a 30% chance of mutating. This helps maintain genetic diversity and prevents premature convergence (when the algorithm doesn't try anything new and gets stuck on the same best solution) .

Maximum fitness evaluations allowed (249900): the algorithm stops once this number of fitness function evaluations is reached.

Elitism ratio (10%): the best 10% paths are preserved by directly sending them to the next generation.

Selection methods (Tournament Selection & Roulette Wheel Selection):

- *Tournament Selection (k=3):* k individuals are randomly selected, and the one with the highest fitness is chosen as a parent. k is also a parameter, with a lower k, there is more diversity since it gives a chance to the not so great solution to be picked.

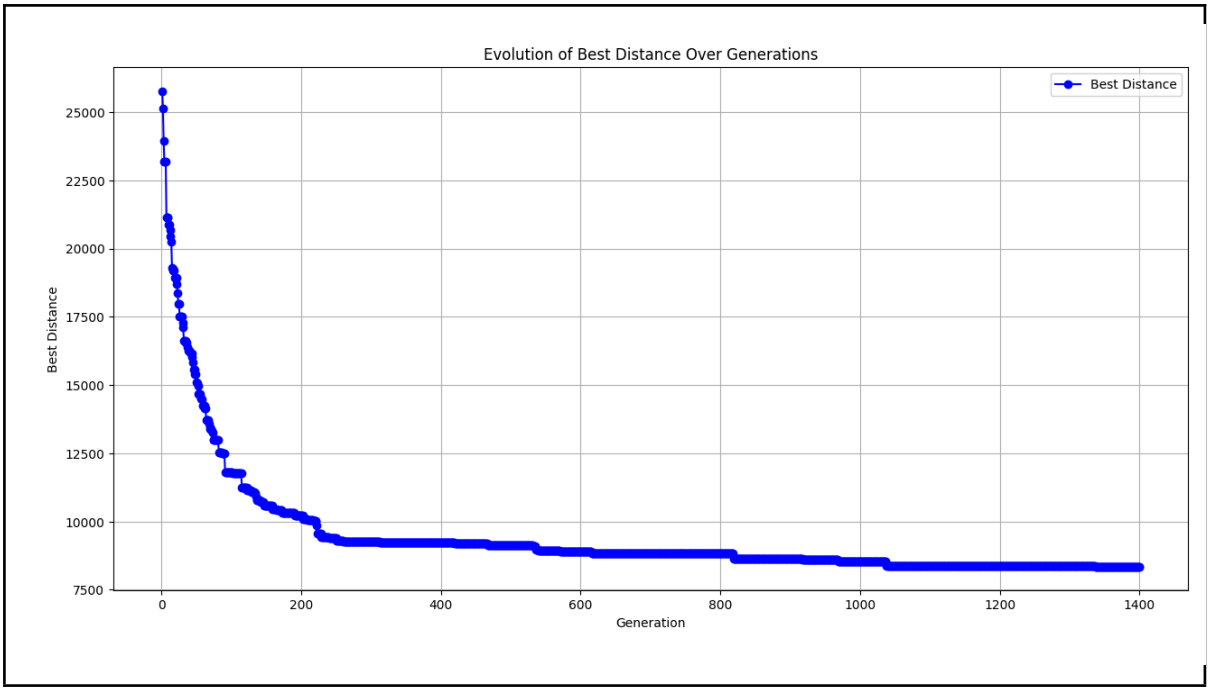
- *Roulette Wheel Selection:* the probability of a path being selected, for crossover, is based on its fitness score by proportion.

- the algorithm switches between the two methods based on a threshold set to (best distance > 13000). This is because both selection methods have a quick and good way of reducing rapidly the total distance either in the beginning or the end of the algorithm, so changing the selection method can help take advantage of that fact.

Crossover method (Ordered Crossover): this method ensures that offspring inherit ordered sequences from their parents. We copy a segment from one parent, and the missing cities are filled in according to the order of the second parent.

Mutation method (Segment Reversal): Instead of only swapping two cities, a whole segment of the path is selected and reversed. This allows to try more combinations.

Task 5 : how does the performance of the population evolve with generations?



The figure above shows the evolution of the best distance according to the generations.

At first, the best distance starts at around 25,000, since they are randomly generated and not optimized.

However, as generations progress, the best distance decreases rapidly.

A sharp decrease in the early generations shows that the algorithm quickly finds significantly better solutions, thanks to tournament selection.

After around 200 generations, the improvement slows down, with best distances stabilizing between 9,000 and 8,000.

This plateau indicates that the algorithm has converged to a near-optimal solution, with minor improvement occurring in later stages, thanks to wheel selection.

Part 2 - The Travelling Salesman Problem with ACO :

Task 1 : What are the important operations of the algorithm ?

The main operations for the **Ant Colony Optimization (ACO)** algorithm are:

1. **Distance and Pheromone Matrix Initialization**
   A distance matrix is built to store the Euclidean distance between each pair of cities. Also, a pheromone matrix is initialized with an equal value (we decided to set it to 10) to represent the amount of pheromone on each path (path between two cities).

2. **Solution Construction**

   From the city (1), each ant chooses, according to probability, a random city and goes to it and repeats the process until it builds a complete tour, where it has visited

all the cities once and then goes back to city (1). At each step, an ant chooses the next city based on a **probability function** that combines pheromone strength (α) and heuristic parameters ($\frac{1}{d(X_i, X_{i+1})}$, β).

$$P(r,s) = \begin{cases} \frac{[\tau_{rs}]^{\alpha} * [\eta_{rs}]^{\beta}}{\sum_{u \in J_k(r)} [\tau_{ru}]^{\alpha} * [\eta_{ru}]^{\beta}} & if\ s \in J_k(r) \\ 0, & in\ other\ case \end{cases}$$

where:

τ_{rs} is the pheromone of the edge e_{rs}

η_{rs} is the heuristic information guiding the path construction.

Greedy case: $\eta_{rs} = \frac{1}{d_{rs}}$ meaning nearest node most desirable

$J_k(r)$ is the set of nodes connected to **r** but not visited by the ant.

Source : Johan Hjorth, Lecture on Swarm intelligence, DVA340, MDU

3. **Pheromone Update**

After each iteration, the pheromone levels are **evaporated** to prevent early convergence to a suboptimal solution.The pheromone matrix is updated based on the fitness of the solutions found by the ants. A path with a shorter distance receives more pheromone reinforcement.

After all ants complete the paths, pheromone will be updated as:

$$\tau_{rs}(t) = (1-p) * \tau_{rs}(t-1) + \Delta\tau_{rs}$$

Pheromone evaporates a small amount after each iteration, with $0 < p < 1$ being an evaporation constant

The density of pheromones laid on the edge (r, s) by the m ants is:

$$\Delta\tau_{rs} = \sum_{k=1}^{m} \Delta\tau_{rs}^{k}$$

where:

$$\Delta\tau_{rs}^{k} = \begin{cases} \frac{1}{C(S_k)}, & if\ the\ ant\ k\ visited\ the\ edge \\ 0, & otherwise \end{cases}$$

$C(S_k)$ is the path length generated by the ant **k**.

Source : Johan Hjorth, Lecture on Swarm intelligence, DVA340, MDU

4. **Best Solution Selection**The best path found by the ants in each generation is compared with the global best path.The algorithm iterates until the number of generations (stopping criteria) is reached.

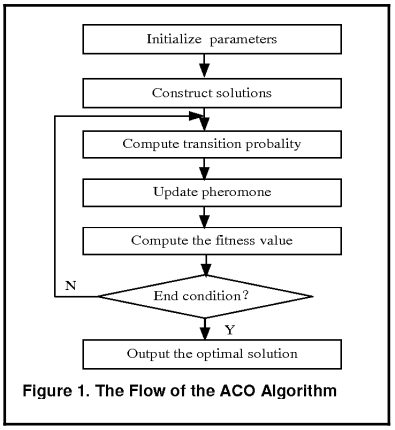
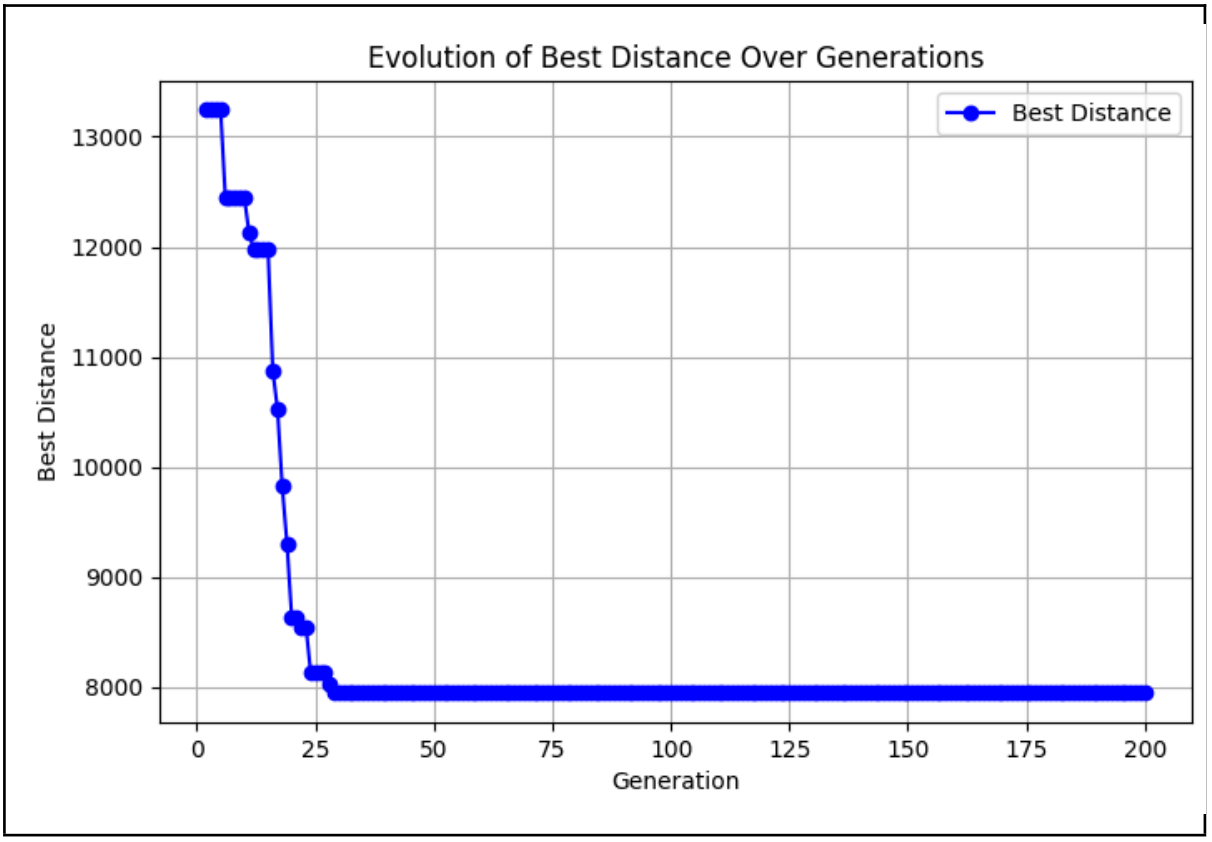


Figure 1. The Flow of the ACO Algorithm

Task 2 : How does the performance of the population evolve with generations?



The best distance initially starts around 13,000, since the paths are not completely random thanks to the heuristic distance.

In the first 20 generations, the best distance rapidly decreases. After reaching approximately 8,000, the improvements slow down, showing convergence to a near-optimal solution.

The plateau, starting from around generation 50, indicates that the algorithm has stabilized, and additional iterations do not significantly improve the result.

Task 3 : How does the results found by this algorithm compare with the results found with GA ?

One key difference between the two algorithms is in the way they converge : ACO has a strong initial improvement for the best distance in the first few generations (because pheromone updates quickly reinforce good paths).
However, this reinforcement can also lead to convergence too quickly if the algorithm always takes the same paths, making it difficult to explore alternative paths.
While on the other hand, GA has a more exploratory approach thanks to the mutation and crossover mechanisms, which allow for continued diversification of paths.

This helps GA avoid local optima more effectively than ACO, but it also means that GA takes longer to reach near-optimal solution.